OS & SP

Name : S LOHITHA **Roll No :** 2520030109

SKILL – 07

1.Skill7.c

lohitha27@LAPTOP-A9F545F(

GNU nano 8.7.1 skill7.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

/* Parse command into arguments */
int parse_command(char *input, char *args[])
{
    int argc = 0;
    char *token = strtok(input, " \t");

    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }

    args[argc] = NULL;
    return argc;
}

/* Display PATH */
void display_path(void)
{
    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH is not set.\n");
        return;
    }

    printf("PATH:\n%s\n", path);
}

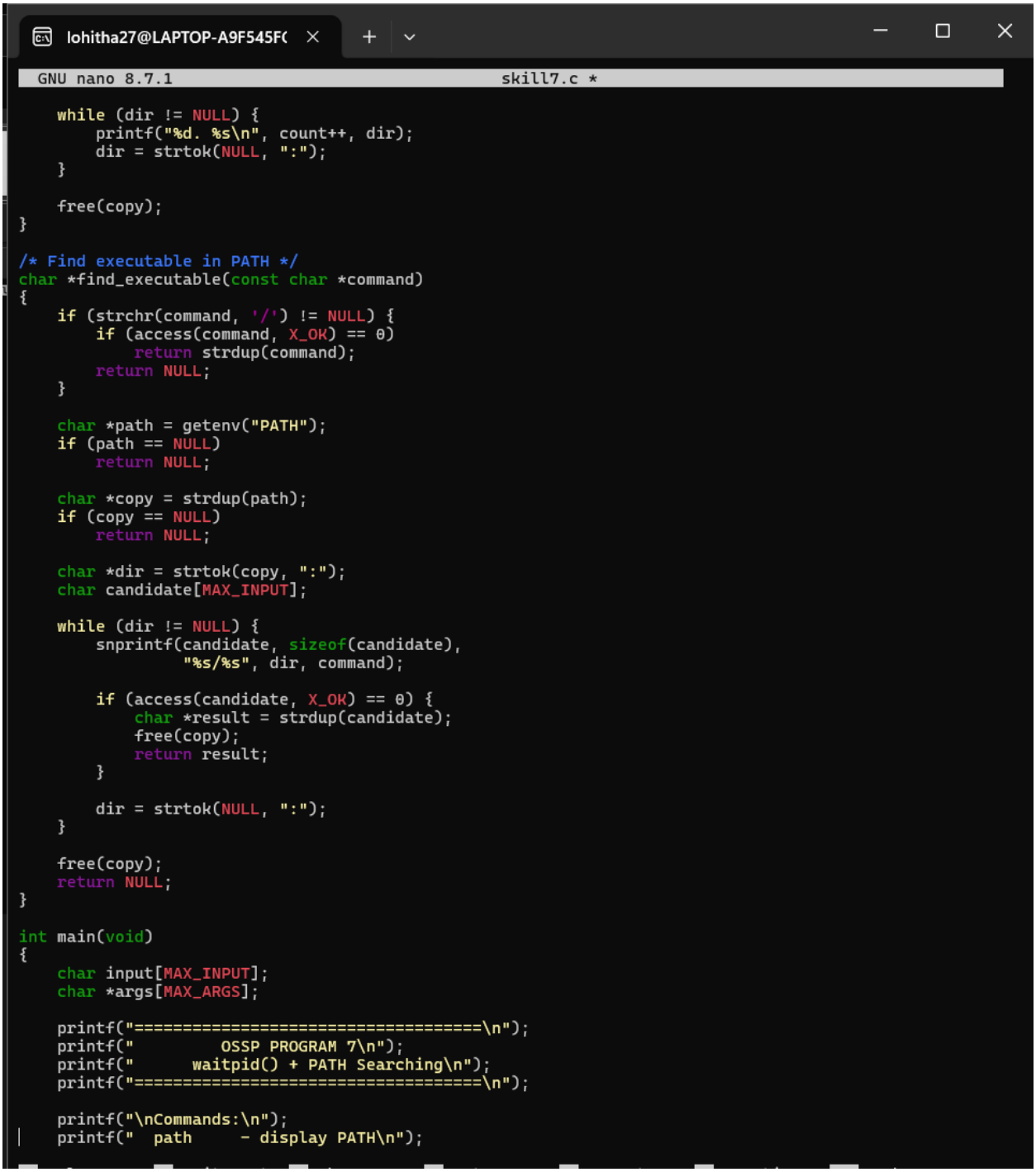
/* Display PATH directories separately */
void display_pathlist(void)
{
    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH is not set.\n");
        return;
    }

    char *copy = strdup(path);
    if (copy == NULL) {
        perror("strdup");
        return;
    }

    printf("PATH directories:\n");

    int count = 1;
    char *dir = strtok(copy, ":");
```


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill7.c *
while (dir != NULL) {
printf("%d. %s\n", count++, dir);
dir = strtok(NULL, ":");
}
free(copy);
}
/* Find executable in PATH */
char *find_executable(const char *command)
{
if (strchr(command, '/') != NULL) {
if (access(command, X_OK) == 0)
return strdup(command);
return NULL;
}
char *path = getenv("PATH");
if (path == NULL)
return NULL;
char *copy = strdup(path);
if (copy == NULL)
return NULL;
char *dir = strtok(copy, ":");
char candidate[MAX_INPUT];
while (dir != NULL) {
snprintf(candidate, sizeof(candidate),
"%s/%s", dir, command);
if (access(candidate, X_OK) == 0) {
char *result = strdup(candidate);
free(copy);
return result;
}
dir = strtok(NULL, ":");
}
free(copy);
return NULL;
}
int main(void)
{
char input[MAX_INPUT];
char *args[MAX_ARGS];
printf("====================================\n");
printf(" OSSP PROGRAM 7\n");
printf(" waitpid() + PATH Searching\n");
printf("====================================\n");
printf("\nCommands:\n");
printf(" path - display PATH\n");


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill7.c *
while (1) {
printf("shell7> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
break;
input[strcspn(input, "\n")] = '\0';
if (input[0] == '\0')
continue;
if (strcmp(input, "exit") == 0)
break;
int argc = parse_command(input, args);
if (argc == 0)
continue;
if (strcmp(args[0], "path") == 0) {
display_path();
continue;
}
if (strcmp(args[0], "pathlist") == 0) {
display_pathlist();
continue;
}
char *executable = find_executable(args[0]);
if (executable == NULL) {
printf("Command not found or not executable: %s\n",
args[0]);
continue;
}
printf("\nExecutable found:\n%s\n", executable);
pid_t pid = fork();
if (pid < 0) {
perror("fork");
free(executable);
continue;
}
if (pid == 0) {
printf("Child process started.\n");
printf("Child PID: %d\n", getpid());
execv(executable, args);
perror("execv");
_exit(EXIT_FAILURE);
} else {



```
            perror("execv");
            _exit(EXIT_FAILURE);
        } else {
            int status;

            printf("Parent PID: %d\n", getpid());
            printf("Child PID: %d\n", pid);
            printf("Parent waiting using waitpid()...\n");

            if (waitpid(pid, &status, 0) == -1) {
                perror("waitpid");
            } else if (WIFEXITED(status)) {
                printf("Child %d exited normally.\n", pid);
                printf("Exit status: %d\n",
                       WEXITSTATUS(status));
            } else {
                printf("Child terminated abnormally.\n");
            }
        }

        free(executable);
        printf("\n");
    }

    printf("Program 7 terminated.\n");
    return 0;
}

^G Help       ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit       ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

OUTPUT :

```
shell6> ^C
lohitha27@LAPTOP-A9F545FG:~/SKILL$ nano skill7.c
lohitha27@LAPTOP-A9F545FG:~/SKILL$ gcc skill7.c -o skill7
lohitha27@LAPTOP-A9F545FG:~/SKILL$ ./skill7
==================================
         OSSP PROGRAM 7
      waitpid() + PATH Searching
==================================

Commands:
  path     - display PATH
  pathlist - display PATH directories
  exit     - quit

shell7> pwd

Executable found:
/usr/bin/pwd
Parent PID: 1353
Child PID: 1354
Parent waiting using waitpid()...
Child process started.
Child PID: 1354
/home/lohitha27/SKILL
Child 1354 exited normally.
Exit status: 0

shell7> valgrind --leak-check=full --show-leak-kinds=all ./program7
Command not found or not executable: valgrind
shell7> pwd

Executable found:
/usr/bin/pwd
Parent PID: 1353
Child PID: 1358
Parent waiting using waitpid()...
Child process started.
Child PID: 1358
/home/lohitha27/SKILL
Child 1358 exited normally.
Exit status: 0

shell7> exit
Program 7 terminated.
lohitha27@LAPTOP-A9F545FG:~/SKILL$
```